

Migrate date functions from Oracle to Amazon RDS for PostgreSQL

by Neha Sharma and Sweta Krishna | on 03 JAN 2024 | in [Advanced \(300\)](#), [RDS for PostgreSQL](#), [Technical How-to](#) | [Permalink](#) | [Comments](#) | [Share](#)
[PostgreSQL](#) is considered one of the top database choices when customers migrate from commercial databases such as Oracle or SQL Server. AWS provides two managed PostgreSQL options: [Amazon Relational Database Service \(Amazon RDS\) for PostgreSQL](#) and [Amazon Aurora PostgreSQL-Compatible Edition](#).

Moving from Oracle to Amazon RDS for PostgreSQL or Amazon Aurora PostgreSQL-Compatible Edition can require updates to the database schema or SQL commands used to access the data. To assist migrations, AWS provides the [AWS Schema Conversion Tool](#) (AWS SCT) to convert application-embedded SQL code and offers a [migration playbook](#) that documents a number of functional mappings between Oracle database and PostgreSQL.

While the tooling helps to automate the migration effort, there are scenarios where a developer needs to intervene and manually convert SQL from Oracle to PostgreSQL. The date and timestamp columns are significantly used in many applications to track the time element of the data. As we migrate from one database to another we need to maintain symmetry between the converted date and time values so that dependent components are not impacted. In this post, we show the difference between Oracle and PostgreSQL date and timestamp related functions and how to convert them to make their result equivalent.

Date-related functions in Oracle and PostgreSQL

Oracle's `DATE` data type is equivalent to `TIMESTAMP WITHOUT TIME ZONE` in PostgreSQL. For demonstration purposes, in the queries of this post we use the `CREATE_DATE` column of `DATE` data type in Oracle and `TIMESTAMP WITHOUT TIME ZONE` in PostgreSQL. The format used in database client setting for Oracle and PostgreSQL is `YYYY-MM-DD HH:MI:SS.SSS`. The code snippets shown in this post are tested in Oracle version 19.0 and [PostgreSQL version 13](#) and later.

Oracle SYSDATE and PostgreSQL NOW() and CURRENT_DATE

In Oracle, `SYSDATE` is used to get the system current date and time, and the format depends on the value of the `NLS_DATE_FORMAT` initialization parameter.

PostgreSQL has a built-in `NOW()` function which returns current date and time till microseconds with time zone, whereas another PostgreSQL function `CURRENT_DATE` gives only the current date. The format of the `DATE` and `TIMESTAMP` depends on the database parameter or database client settings. Both of these functions are not equivalent to `SYSDATE` function of Oracle. Oracle's `SYSDATE` output is of `DATE` data type whereas PostgreSQL `NOW()` is of `TIMESTAMP` datatype and `CURRENT_DATE` is of `DATE` data type. For more information, refer to [Oracle and PostgreSQL system dates and timestamps are different](#).

The following table shows the difference between Oracle's `SYSDATE` and PostgreSQL `CURRENT_DATE`.

Oracle	PostgreSQL
<pre>SELECT SYSDATE FROM DUAL ; SYSDATE ----- 2023-07-07 09:18:03.000</pre>	<pre>SELECT CURRENT_DATE ; current_date ----- 2023-07-07</pre>
	<pre>SELECT NOW() ; current_date ----- 2023-07-07 09:23:45.729985+02</pre>

You can cast PostgreSQL's `CURRENT_TIMESTAMP` to `TIMESTAMP WITHOUT TIME ZONE` to get a similar result as Oracle's `SYSDATE`.

Oracle	PostgreSQL
<pre>SELECT SYSDATE FROM DUAL ; SYSDATE ----- 2023-07-07 09:23:45.000</pre>	<pre>SELECT CURRENT_TIMESTAMP::TIMESTAMP WITHOUT TIME ZONE ; current_timestamp ----- 2023-07-07 09:23:45.729</pre>

TRUNC in Oracle and DATE_TRUNC in PostgreSQL

In Oracle, you can use `TRUNC` with the `DATE` data type to truncate the date to a specific level of precision. In PostgreSQL, you can use `CAST` or `DATE_TRUNC` to truncate the date or timestamp column. `CAST` is used to convert one data type to another and `DATE_TRUNC` is used to truncate `TIMESTAMP` or `INTERVAL` column.

Oracle	PostgreSQL
Function: <code>TRUNC (DATE)</code>	Function: <code>CAST</code> , <code>DATE_TRUNC</code>
The <code>TRUNC</code> function is used to get the date with the time portion of the day truncated to a specific unit of measure: <pre>TRUNC(<date field> [, datepart])</pre> The <code>date_part</code> includes <code>DD</code>, <code>MM</code>, <code>YYYY</code>, <code>HH</code>, <code>MI</code> and so on.	<code>CAST</code> : A cast specifies how to perform a conversion between two data types. Here, a cast is used to convert the date data type to a timestamp without a time zone. <code>DATE_TRUNC</code> : Extracts a <code>TIMESTAMP</code> or <code>INTERVAL</code> column and truncates to a specific level of precision: <code>DATE_TRUNC(date_part, Field)</code> The <code>date_part</code> includes <code>day</code> , <code>month</code> , <code>year</code> , <code>hour</code> , <code>minute</code> , and so on.
<pre>SELECT TRUNC(SYSDATE) AS "DATE_ORACLE" FROM DUAL; DATE_ORACLE ----- 2023-07-06 00:00:00.000</pre>	<pre>SELECT CAST(CURRENT_DATE AS TIMESTAMP) AS "DATE_POSTGRESQL"; DATE_POSTGRESQL ----- 2023-07-06 00:00:00.000</pre> <pre>SELECT date_trunc('day', CURRENT_DATE)::timestamp AS "DATE_POSTGRESQL"; DATE_POSTGRESQL ----- 2023-07-06 00:00:00.000</pre>

TRUNC BY DAY/MONTH/YEAR in Oracle and PostgreSQL

The truncate function with `DATE` or `TIMESTAMP` datatype column is used to round transaction timestamps to desired granularity. For reporting application this function helps to create time based aggregation like daily, weekly or monthly summaries of data in Oracle and PostgreSQL database.

The `TRUNC (<DateField>, <fmt>) (date)` function in Oracle returns `DATE` with the time portion of the day truncated to the unit specified for format model (`fmt`): `'DD'/'MONTH'/'YEAR'`. The value returned is always of data type `DATE`. Similar functionality can be achieved using `DATE_TRUNC()` function in PostgreSQL. The `date_trunc()` is used to truncate a DATE, TIME, or TIMESTAMP to the specified precision.

The `TRUNC (<DateField>,'DD')` function in Oracle and `DATE_TRUNC('DAY', <DateField>)` function in PostgreSQL truncate the date column to day '`YYYY-MM-DD`' and return time portion as '`00:00:00.000`'. In the following example, the `CREATE_DATE` column is of the `DATE` data type in Oracle and `TIMESTAMP WITHOUT TIME ZONE` data type in PostgreSQL. Oracle's `TRUNC` function with argument '`DD`' truncates the date column's time portion from `12:53:19.000` to `00:00:00.000`. It can be converted in PostgreSQL using `DATE_TRUNC` with '`DAY`' as an argument.

The `TRUNC(<DateField>,'MONTH')` function in Oracle and `DATE_TRUNC('MONTH', <DateField>)` function in PostgreSQL truncate the date column to month part. PostgreSQL truncate the date column to day '`YYYY-MM-DD`' and exclude time portion. In the following example, Oracle's `TRUNC` function with argument '`MONTH`' truncates the date column from the twenty-first day of the month to the first day of the month. It can be converted in PostgreSQL using `DATE_TRUNC` with '`MONTH`' as an argument.

The `TRUNC(<DateField>,'YEAR')` function in Oracle and `DATE_TRUNC('YEAR', <DateField>)` function in PostgreSQL truncate the date column to year part.

In the following example, Oracle's `TRUNC` function with argument '`YEAR`' truncates the date column to the first day of the year. It can be converted in PostgreSQL using `DATE_TRUNC` with '`YEAR`' as an argument.

Oracle	PostgreSQL
Function: <code>TRUNC</code> for <code>DAY</code>	Function: <code>DATE_TRUNC</code> for <code>DAY</code>

• TRUNC for MONTH • TRUNC for YEAR	• DATE_TRUNC for MONTH • DATE_TRUNC for YEAR
<pre>SELECT CREATE_DATE as "CREATE_DATE", TRUNC(CREATE_DATE, 'DD') AS "TRUNC_BY_DAY" FROM TAB_DATE; CREATE_DATE TRUNC_BY_DAY ----- 2018-03-21 12:53:19.000 2018-03-21 00:00:00.000</pre>	<pre>SELECT CREATE_DATE as "CREATE_DATE", DATE_TRUNC('DAY',CREATE_DATE) AS "TRUNC_BY_DAY" FROM TAB_DATE; CREATE_DATE TRUNC_BY_DAY ----- 2018-03-21 12:53:19.000 2018-03-21 00:00:00.000</pre>
<pre>SELECT CREATE_DATE as "CREATE_DATE", TRUNC(CREATE_DATE, 'MONTH') AS "TRUNC_BY_MONTH" FROM TAB_DATE; CREATE_DATE TRUNC_BY_MONTH ----- 2018-03-21 12:53:19.000 2018-03-01 00:00:00.000</pre>	<pre>SELECT CREATE_DATE as "CREATE_DATE", DATE_TRUNC('MONTH',CREATE_DATE) AS "TRUNC_BY_MONTH" FROM TAB_DATE; CREATE_DATE TRUNC_BY_MONTH ----- 2018-03-21 12:53:19.000 2018-03-01 00:00:00.000</pre>
<pre>SELECT CREATE_DATE as "CREATE_DATE", TRUNC(CREATE_DATE, YEAR) AS "TRUNC_BY_YEAR" FROM TAB_DATE; CREATE_DATE TRUNC_BY_YEAR ----- 2018-03-21 12:53:19.000 2018-01-01 00:00:00.000</pre>	<pre>SELECT CREATE_DATE as "CREATE_DATE", DATE_TRUNC('YEAR', CREATE_DATE) AS "TRUNC_BY_YEAR" FROM TAB_DATE; CREATE_DATE TRUNC_BY_YEAR ----- 2018-03-21 12:53:19.000 2018-01-01 00:00:00.000</pre>

TO_DATE in Oracle and PostgreSQL

The `TO_DATE` function is used to convert the character data type to date. It gives the result in a different format in Oracle and PostgreSQL. In Oracle the result is in `'YYYY-MM-DD 00:00:00.000'` format, whereas in PostgreSQL it is in this format `'YYYY-MM-DD'` . Therefore explicit typecasting is required.

The following table illustrates using `TO_DATE` with format `'YYYY-MM-DD'` in Oracle and PostgreSQL and the difference in the result.

Oracle	PostgreSQL
Function: <code>TO_DATE</code>	Function: <code>TO_DATE</code>
<pre>SELECT TO_DATE('2023-06-26', 'YYYY-MM-DD') AS DATE_ORACLE FROM DUAL; DATE_ORACLE ----- 2023-06-26 00:00:00.000</pre>	<pre>SELECT TO_DATE('2023-06-26', 'YYYY-MM-DD') AS "DATE_POSTGRESQL"; DATE_POSTGRESQL ----- 2023-06-26</pre>

To convert `TO_DATE(<DATE_FIELD>, 'YYYY-MM-DD')` from Oracle to the equivalent in PostgreSQL, typecasting is required: `TO_DATE(<DATE_FIELD>, 'YYYY-MM-DD')::TIMESTAMP(0)` or `TO_DATE(<DATE_FIELD>, 'YYYY-MM-DD')::TIMESTAMP WITHOUT TIME ZONE)` .

In the following example typecasting of `TO_DATE(<DATE_FIELD>, 'YYYY-MM-DD')` to `TIMESTAMP(0)` is done in PostgreSQL.

Oracle	PostgreSQL

<pre>SELECT TO_DATE('2023-06-26', 'YYYY-MM-DD') AS DATE_ORACLE FROM DUAL; DATE_ORACLE ----- 2023-06-26 00:00:00.000</pre>	<pre>SELECT TO_DATE('2023-06-26', 'YYYY-MM-DD') ::TIMESTAMP(0) AS "DATE_POSTGRESQL"; DATE_POSTGRESQL ----- 2023-06-26 00:00:00.000</pre>
---	--

In the following example typecasting of `TO_DATE(<DATE_FIELD>, 'YYYY-MM-DD')` to `TIMESTAMP WITHOUT TIME ZONE` is done in PostgreSQL.

Oracle	PostgreSQL
<pre>SELECT TO_DATE('2023-06-26', 'YYYY-MM-DD') AS DATE_ORACLE FROM DUAL; DATE_ORACLE ----- 2023-06-26 00:00:00.000</pre>	<pre>SELECT TO_DATE('2023-06-26', 'YYYY-MM-DD') ::TIMESTAMP WITHOUT TIME ZONE AS "DATE_POSTGRESQL"; DATE_POSTGRESQL ----- 2023-06-26 00:00:00.000</pre>

The following table illustrates using `TO_DATE` with format `'YYYY-MM-DD HH24:MI:SS'`.

The `TO_DATE` function in Oracle and PostgreSQL gives different results. Oracle returns the result in `'YYYY-MM-DD HH24:MI:SS'` format, whereas PostgreSQL returns it in `'YYYY-MM-DD'`.

Oracle	PostgreSQL
Function: <code>TO_DATE(<Date_Field>,'YYYY-MM-DD HH24:MI:SS')</code> <pre>SELECT TO_DATE('2023-06-26 13:15:53', 'YYYY-MM-DD HH24:MI:SS') AS DATE_ORACLE FROM DUAL; DATE_ORACLE ----- 2023-06-26 13:15:53.000</pre>	Function: <code>TO_DATE(<Date_Field>,'YYYY-MM-DD HH24:MI:SS')</code> <pre>SELECT TO_DATE('2023-06-26 13:15:53', 'YYYY-MM-DD HH24:MI:SS') AS "DATE_POSTGRESQL"; DATE_POSTGRESQL ----- 2023-06-26</pre>

To convert the `TO_DATE(<DATE_FIELD>, 'YYYY-MM-DD HH24:MI:SS')` function in Oracle to PostgreSQL, `TO_TIMESTAMP` is required, along with typecasting: `TO_TIMESTAMP(<Date_Field>, 'YYYY-MM-DD HH24:MI:SS')::TIMESTAMP(0)` or `TO_TIMESTAMP(<Date_Field>, 'YYYY-MM-DD HH24:MI:SS')::TIMESTAMP WITHOUT TIME ZONE`.

In the following example typecasting of `TO_DATE(<DATE_FIELD>, 'YYYY-MM-DD')` to `TIMESTAMP(0)` is done in PostgreSQL.

Oracle	PostgreSQL
<pre>SELECT TO_DATE('2023-06-26 13:15:53', 'YYYY-MM-DD HH24:MI:SS') AS DATE_ORACLE FROM DUAL; DATE_ORACLE ----- 2023-06-26 13:15:53.000</pre>	<pre>SELECT TO_TIMESTAMP('2023-06-26 13:15:53', 'YYYY-MM-DD HH24:MI:SS')::T AS "DATE_POSTGRESQL"; DATE_POSTGRESQL ----- 2023-06-26 13:15:53.000</pre>

In the following example typecasting of `TO_DATE(<DATE_FIELD>, 'YYYY-MM-DD')` to `TIMESTAMP WITHOUT TIME ZONE` is done in PostgreSQL.

Oracle	PostgreSQL
<pre>SELECT TO_DATE('2023-06-26 13:15:53', 'YYYY-MM-DD HH24:MI:SS') AS DATE_ORACLE FROM DUAL; DATE_ORACLE ----- 2023-06-26 13:15:53.000</pre>	<pre>SELECT TO_TIMESTAMP('2023-06-26 13:15:53', 'YYYY-MM-DD HH24:MI:SS')::TIMESTAMP WITHOUT TIME ZONE AS "DATE_POSTGRESQL"; DATE_POSTGRESQL ----- 2023-06-26 13:15:53.000</pre>

Add INTEGER to DATE data type in Oracle and PostgreSQL

In scenarios where the number of days are added to a `DATE` column, result matches in both Oracle and PostgreSQL. However, when number of days are added to a `TIMESTAMP` in PostgreSQL it gives an error.

To convert such queries from Oracle to PostgreSQL, `INTERVAL` should be added to a timestamp column.

Oracle	PostgreSQL
<code>DATE + INTEGER = DATE</code>	<code>TIMESTAMP WITHOUT TIME ZONE + INTERVAL = TIMESTAMP WITHOUT TIME ZONE</code>
<pre>SELECT CREATE_DATE AS "CREATE_DATE", CREATE_DATE + 2 AS "NEW_DATE" FROM TAB_DATE; CREATE_DATE NEW_DATE ----- 2018-03-21 12:53:19.000 2018-03-23 12:53:19.000</pre>	<pre>SELECT CREATE_DATE AS "CREATE_DATE", CREATE_DATE + INTERVAL '2' DAY AS "NEW_DATE" FROM TAB_DATE; CREATE_DATE NEW_DATE ----- 2018-03-21 12:53:19.000 2018-03-23 12:53:19.000</pre>

PostgreSQL doesn't allow an integer to be added to a `TIMESTAMP` column. It gives an error as shown below. To resolve it, add `INTERVAL` to a `TIMESTAMP` column.

```
SELECT CREATE_DATE AS "CREATE_DATE", CREATE_DATE + 2 AS "NEW_DATE"
FROM TAB_DATE;
```

SQL Error [42883]: ERROR: operator does not exist: timestamp without time zone + integer

Hint: No operator matches the given name and argument types. You might need to add explicit type casts.

Add an INTEGER column to the DATE data type column in Oracle and PostgreSQL

Sometimes number of days can be stored in database as `INTEGER` column and there can be arithmetic operation between `DATE` column and an `INTEGER` column. In Oracle `INTEGER` column can be added to a `DATE` column but in PostgreSQL to add `INTEGER` column to a `TIMESTAMP` column the `INTEGER` column must be converted into `INTERVAL` and then added to a `TIMESTAMP` column.

In this example, we multiply the `INTEGER` column with `'1' DAY INTERVAL` and then add this interval value to a `DATE` or `TIMESTAMP WITHOUT TIME ZONE` column.

Oracle	PostgreSQL
<code>DATE + INTEGER (COLUMN) = DATE</code>	<code>TIMESTAMP WITHOUT TIME ZONE + INTERVAL '1' DAY * INTEGER (COLUMN) = TIMESTAMP WITHOUT TIME ZONE</code>

<pre>SELECT CREATE_DATE AS " DATE", CREATE_DATE + type_id AS "NEW_DATE" FROM TAB_DATE; DATE NEW_DATE ----- ----- 2018-01-10 14:04:44.000 2018-01-11 14:04:44.000</pre>	<pre>SELECT CREATE_DATE AS "DATE", CREATE_DATE + INTERVAL '1' day * type_id AS "NEW_DATE" FROM TAB_DATE; DATE NEW_DATE ----- ----- 2018-01-10 14:04:44.000 2018-01-11 14:04:44.000</pre>
---	---

If you don't follow the previous steps, you will get an error because PostgreSQL doesn't allow an integer to be added to a `DATE` or `TIMESTAMP` column.

<pre>SELECT CREATE_DATE AS " DATE", CREATE_DATE + type_id AS "NEW_DATE" FROM TAB_DATE;</pre>	<pre>----- SQL Error [42883]: ERROR: operator does not exist: timestamp without time zone + numeric Hint: No operator matches the given name and argument types. You might need to add explicit type casts. -----</pre>
--	---

Similarly, it gives an error while adding a `DATE` column with the `NUMERIC` column.

<pre>SELECT CREATE_DATE AS "DATE", CREATE_DATE::date + type_id AS "NEW_DATE" FROM TAB_DATE;</pre>	<pre>----- SQL Error [42883]: ERROR: operator does not exist: date + numeric Hint: No operator matches the given name and argument types. You might need to add explicit type casts. -----</pre>
---	--

Date difference in Oracle and PostgreSQL

When two date columns are subtracted in Oracle, the result is in `NUMBER` , whereas it's `INTERVAL` in PostgreSQL for timestamp columns.

Oracle	PostgreSQL
<pre>SELECT EXPIRY_DATE, CREATE_DATE , EXPIRY_DATE - CREATE_DATE AS "DATE_DIFF" FROM TAB_DATE; EXPIRY_DATE CREATE_DATE DATE_DIFF ----- ----- ----- 2018-01-11 23:59:00.000 2018-01-10 14:04:44.000 1.4126851852</pre>	<pre>SELECT EXPIRY_DATE, CREATE_DATE , EXPIRY_DATE - CREATE_DATE AS "DATE_DIFF" FROM TAB_DATE; EXPIRY_DATE CREATE_DATE DATE_DIFF ----- ----- ----- 2018-01-11 23:59:00.000 2018-01-10 14:04:44.000 1 day</pre>

In PostgreSQL, to calculate the date difference in days where date datatype is timestamp and related datatype, we can use the `EXTRACT` with `epoch` function to get the difference between two dates in numeric. Also, `EXTRACT` can be used with date of interval datatype, convert it into seconds, and then divide by total number of seconds in a day, which is `24*60*60` .

Oracle	PostgreSQL
<pre>Date 1 - Date 2</pre>	<pre>Function: EXTRACT in PostgreSQL retrieves a field such as a year, month, a</pre>

	Function: <code>EPOCH</code> time represents the total number of seconds elapsed. To extract the current or specific time, the <code>EXTRACT()</code> function is used with the <code>EPOCH</code> function. <pre>EXTRACT (EPOCH FROM DateTime);</pre>
<pre>SELECT EXPIRY_DATE, CREATE_DATE , EXPIRY_DATE - CREATE_DATE AS "DATE_DIFF" FROM TAB_DATE; EXPIRY_DATE CREATE_DATE DATE_DIFF ----- 2018-01-11 23:59:00.000 2018-01-10 14:04:44.000 1.4126851852</pre>	<pre>SELECT EXPIRY_DATE AS "EXPIRY_DATE", CREATE_DATE as "CRE (EXTRACT (DAY FROM (expiry_date-create_date))*24*60*60+ EXTRACT (HOUR FROM (expiry_date-create_date))*60*60+ EXTRACT (MINUTE FROM (expiry_date-create_date))*60+ EXTRACT (SECOND FROM (expiry_date-create_date)))/(24*60* FROM TAB_DATE; EXPIRY_DATE CREATE_DATE DATE_DIFF_IN ----- 2018-01-11 23:59:00.000 2018-01-10 14:04:44.000 1.4126</pre>
<pre>SELECT EXPIRY_DATE, CREATE_DATE , EXPIRY_DATE - CREATE_DATE AS "DATE_DIFF" FROM TAB_DATE; EXPIRY_DATE CREATE_DATE DATE_DIFF ----- 2018-01-11 23:59:00.000 2018-01-10 14:04:44.000 1.4126851852</pre>	<pre>SELECT EXPIRY_DATE , CREATE_DATE, EXTRACT('epoch' FROM EXPIRY_DATE - CREATE_DATE)/ (24*60* AS "DATE_DIFF_INT" from TAB_DATE; EXPIRY_DATE CREATE_DATE DATE_DIFF_IN ----- 2018-01-11 23:59:00.000 2018-01-10 14:04:44.000 1.4126</pre>

Date difference using TRUNC in Oracle

In Oracle, when `TRUNC` is used with date columns to get the date difference, the result is `INTEGER` value. This can be achieved in PostgreSQL using the `CAST` or `DATE_PART` function.

Oracle	PostgreSQL
Function: <code>TRUNC</code> <pre>SELECT TRUNC(SYSDATE+93)- TRUNC(SYSDATE) AS "DATE_DIFF_IN_DAY" FROM DUAL; DATE_DIFF_IN_DAY ----- 93</pre>	<pre>SELECT (CURRENT_DATE +93) - CURRENT_DATE AS "DATE_DIFF_IN_DAY"; DATE_DIFF_IN_DAY ----- 93</pre>

Oracle	PostgreSQL
Function: <code>TRUNC</code>	Function: <code>DATE_PART</code> is used to retrieve a specific part (like a month, year, h minutes) from a date or time: <pre>DATE_PART(field, source)</pre> The field determines which date/time part will be extracted or pulled out from given source.
<pre>SELECT EXPIRY_DATE , CREATE_DATE ,</pre>	<pre>SELECT EXPIRY_DATE AS "EXPIRY_DATE", CREATE_DATE AS "CREATE_DATE",</pre>

<pre>TRUNC(EXPIRY_DATE) - TRUNC(CREATE_DATE) AS "DATE_DIFF" FROM TAB_DATE; EXPIRY_DATE CREATE_DATE DATE_DIFF ----- 2018-01-11 23:59:00.000 2018-01-10 14:04:44.000 1</pre>	<pre>DATE_PART('DAY',EXPIRY_DATE - CREATE_DATE) AS "DATE_DIFF" FROM TAB_DATE; EXPIRY_DATE CREATE_DATE DATE_DIFF ----- 2018-01-11 23:59:00.000 2018-01-10 14:04:44.000 1</pre>
--	---

Date difference in months in Oracle and PostgreSQL

In Oracle, `MONTHS_BETWEEN` gives the date difference in `NUMBER`, whereas in PostgreSQL, the `AGE` function gives the date difference in `INTERVAL`.

Oracle	PostgreSQL
Function: <code>MONTHS_BETWEEN</code> The <code>MONTHS_BETWEEN()</code> function is used to get the number of months between dates (date1 and date2): <pre>MONTHS_BETWEEN(date1, date2)</pre>	Function: <code>AGE</code> The <code>AGE()</code> function accepts two <code>TIMESTAMP</code> values. It subtracts the second argument from the first one and returns <code>INTERVAL</code> as a result: <pre>AGE(timestamp,timestamp)</pre>
<pre>SELECT MONTHS_BETWEEN(SYSDATE+93,SYSDATE) AS "MNTH_BETWEEN" FROM DUAL; MNTH_BETWEEN ----- 3.0322580645</pre>	<pre>SELECT AGE(CURRENT_DATE + 93, CURRENT_DATE) AS "MNTH_BETWEEN"; MNTH_BETWEEN ----- 3 mons 1 day</pre>

The `MONTHS_BETWEEN` function along with `TRUNC` gives the result `INTEGER`. This can be converted in PostgreSQL using `DATE_PART` and the `AGE` function.

Oracle	PostgreSQL
Function: <code>MONTHS_BETWEEN</code> <pre>SELECT TRUNC(MONTHS_BETWEEN(SYSDATE+93,SYSDATE)) AS AGE FROM DUAL; AGE ----- 3</pre>	Function: <code>AGE</code> <pre>SELECT DATE_PART('MONTH',AGE(CURRENT_DATE + 93, CURRENT_DATE)) as "AGE"; AGE ----- 3</pre>

In PostgreSQL, you can use `AGE` with the `EXTRACT` function to match Oracle's `MONTHS_BETWEEN` result. It converts the AGE function's output from `INTERVAL` to `NUMERIC`.

Oracle	PostgreSQL
Function: <code>MONTHS_BETWEEN</code> <pre>SELECT MONTHS_BETWEEN (SYSDATE+93,SYSDATE) AS "NUM_OF_MONTHS" FROM DUAL; NUM_OF_MONTHS</pre>	Function: <code>AGE</code> <pre>SELECT (EXTRACT(YEAR from AGE(CURRENT_DATE + 93, CURRENT_DATE))*12 + EXTRACT(MONTH from AGE(CURRENT_DATE + 93, CURRENT_DATE)) + (EXTRACT(DAY from AGE(CURRENT_DATE + 93, CURRENT_DATE))/31))as "NUM_OF_MONTHS"; NUM_OF_MONTHS</pre>

----- 3.0322580645	----- 3.0322580645
-----------------------------------	-----------------------------------

Convert the NUMTODSINTERVAL function in Oracle to PostgreSQL

In Oracle `NUMTODSINTERVAL` function is used to convert a `NUMBER` value to an `INTERVAL` . It converts a given `NUMBER` value to an `INTERVAL DAY TO SECOND` literal. In `NUMTODSINTERVAL(n, 'interval_unit')` , the `n` argument is the input value and '`interval_unit`' should be one of the following value: '`DAY`' , '`HOURL`' , '`MINUTE`' , or '`SECOND`' .

For example:

- `NUMTODSINTERVAL(90, 'SECOND')` changes `90 seconds` into `0 0:1:30.0` , which is equal to `1 minute` and `30 seconds` .
- `NUMTODSINTERVAL(90, 'MINUTE')` changes `90 minutes` into `0 1:30:0.0` , which is equal to `1 hour` and `30 minutes` .
- `NUMTODSINTERVAL(90, 'HOURL')` changes `90 hours` into `3 18:0:0.0` , which is equal to `3 days` and `18 hours` .

<pre>SELECT NUMTODSINTERVAL(90, 'SECOND') AS "SEC_INTERVAL" , NUMTODSINTERVAL(90, 'MINUTE') AS "MIN_INTERVAL", NUMTODSINTERVAL(90, 'HOURL') AS "HOURL_INTERVAL" FROM DUAL;</pre>		
SEC_INTERVAL	MIN_INTERVAL	HOURL_INTERVAL
-----	-----	-----
0 0:1:30.0	0 1:30:0.0	3 18:0:0.0

In the following example we will convert a number value to an `INTERVAL DAY TO SECOND` literal.

Oracle	PostgreSQL																								
Function: <code>NUMTODSINTERVAL</code> <code>90 seconds</code> is converted into <code>1 minute</code> and <code>30 seconds</code> using <code>NUMTODSINTERVAL(90, 'SECOND')</code> <code>65 minutes</code> is converted into <code>1 hour</code> and <code>5 minutes</code> using <code>NUMTODSINTERVAL(65, 'MINUTE')</code> <code>25 hour</code> is converted into <code>1 day</code> and <code>1 hour</code> using <code>NUMTODSINTERVAL(25, 'HOURL')</code>	Function: <code>INTERVAL</code> PostgreSQL has <code>INTERVAL</code> literal for adding <code>INTERVAL</code> into <code>DAT</code> <code>90 seconds</code> is converted into <code>1 minute</code> and <code>30 seconds</code> usi <code>INTERVAL '90' SECOND</code> . <code>65 minutes</code> is converted into <code>1 hour</code> and <code>5 minutes</code> using <code>INTERVAL '65' MINUTE</code> . <code>25 hour</code> is converted into <code>1 day</code> and <code>1 hour</code> using <code>INTERVA</code>																								
<pre>SELECT CREATE_DATE AS "DATE", CREATE_DATE + NUMTODSINTERVAL(90, 'SECOND') AS "SEC_INTERVAL_DATE", CREATE_DATE + NUMTODSINTERVAL(65, 'MINUTE') AS "MIN_INTERVAL_DATE", CREATE_DATE + NUMTODSINTERVAL(25, 'HOURL') AS "HOURL_INTERVAL_DATE" FROM TAB_DATE;</pre> <table><tr><td>DATE</td><td> SEC_INTERVAL_DATE</td></tr><tr><td> MIN_INTERVAL_DATE</td><td> HOURL_INTERVAL_DATE</td></tr><tr><td>-----</td><td>-----</td></tr><tr><td>-----</td><td>-----</td></tr><tr><td>2018-03-21 12:53:19.000</td><td> 2018-03-21 12:54:49.000</td></tr><tr><td> 2018-03-21 13:58:19.000</td><td> 2018-03-22 13:53:19.000</td></tr></table>	DATE	SEC_INTERVAL_DATE	MIN_INTERVAL_DATE	HOURL_INTERVAL_DATE	-----	-----	-----	-----	2018-03-21 12:53:19.000	2018-03-21 12:54:49.000	2018-03-21 13:58:19.000	2018-03-22 13:53:19.000	<pre>SELECT CREATE_DATE AS "DATE", CREATE_DATE + INTERVAL '90' SECOND AS "SEC_INTERVA CREATE_DATE + INTERVAL '65' MINUTE AS "MIN_INTERVA CREATE_DATE + INTERVAL '25' HOURL AS "HOURL_INTERVAL FROM TAB_DATE;</pre> <table><tr><td>DATE</td><td> SEC_INTERVAL_DATE</td></tr><tr><td> MIN_INTERVAL_DATE</td><td> HOURL_INTERVAL_DATE</td></tr><tr><td>-----</td><td>-----</td></tr><tr><td>-----</td><td>-----</td></tr><tr><td>2018-03-21 12:53:19.000</td><td> 2018-03-21 12:54:49.000</td></tr><tr><td> 2018-03-21 13:58:19.000</td><td> 2018-03-22 13:53:19.0</td></tr></table>	DATE	SEC_INTERVAL_DATE	MIN_INTERVAL_DATE	HOURL_INTERVAL_DATE	-----	-----	-----	-----	2018-03-21 12:53:19.000	2018-03-21 12:54:49.000	2018-03-21 13:58:19.000	2018-03-22 13:53:19.0
DATE	SEC_INTERVAL_DATE																								
MIN_INTERVAL_DATE	HOURL_INTERVAL_DATE																								
-----	-----																								
-----	-----																								
2018-03-21 12:53:19.000	2018-03-21 12:54:49.000																								
2018-03-21 13:58:19.000	2018-03-22 13:53:19.000																								
DATE	SEC_INTERVAL_DATE																								
MIN_INTERVAL_DATE	HOURL_INTERVAL_DATE																								
-----	-----																								
-----	-----																								
2018-03-21 12:53:19.000	2018-03-21 12:54:49.000																								
2018-03-21 13:58:19.000	2018-03-22 13:53:19.0																								

Convert ADD_MONTHS in ORACLE to PostgreSQL

Oracle	PostgreSQL
--------	------------

Function: <code>ADD_MONTHS</code> The <code>ADD_MONTHS()</code> function returns a date with a specified number of months added: <pre>ADD_MONTHS(date1, number_months)</pre>	Function: <code>INTERVAL</code> PostgreSQL allows us to specify intervals when adding to dates: <pre>+ INTERVAL 'n' MONTH</pre>
<pre>SELECT CREATE_DATE AS "DATE", ADD_MONTHS(CREATE_DATE,4) AS "NEW_DATE" FROM TAB_DATE; DATE NEW_DATE ----- ----- 2018-03-21 12:53:19.000 2018-07-21 12:53:19.000</pre>	<pre>SELECT CREATE_DATE AS "DATE", CREATE_DATE + INTERVAL '4' MONTH AS "NEW_DATE" FROM TAB_DATE; DATE NEW_DATE ----- ----- 2018-03-21 12:53:19.000 2018-07-21 12:53:19.000</pre>

Convert date series in Oracle to PostgreSQL

In Oracle, you can use `CONNECT BY LEVEL` to generate series. PostgreSQL has the `GENERATE_SERIES` function, which allows you to generate a set of data starting at one point, ending at another point, and optionally set the incrementing value.

Oracle	PostgreSQL
Function: <code>CONNECT BY</code> with <code>LEVEL</code>	Function: <code>GENERATE_SERIES</code> with <code>LEVEL</code> : <pre>generate_series([start], [stop], [{optional}step/interval])</pre>
<pre>SELECT (TO_DATE('2023/07/01', 'YYYY/MM/DD') + LEVEL - 1) "DATE", LEVEL FROM DUAL CONNECT BY LEVEL <= 10; DATE LEVEL -----+----- 2023-07-01 00:00:00 1 2023-07-02 00:00:00 2 2023-07-03 00:00:00 3 2023-07-04 00:00:00 4 2023-07-05 00:00:00 5 2023-07-06 00:00:00 6 2023-07-07 00:00:00 7 2023-07-08 00:00:00 8 2023-07-09 00:00:00 9 2023-07-10 00:00:00 10</pre>	<pre>SELECT "DATE", LEVEL FROM GENERATE_SERIES ('2023-07-01'::TIMESTAMP, '2023-07-10'::TIMESTAMP, INTERVAL '1 DAY') WITH ORDINALITY AS T ("DATE",LEVEL); DATE LEVEL -----+----- 2023-07-01 00:00:00 1 2023-07-02 00:00:00 2 2023-07-03 00:00:00 3 2023-07-04 00:00:00 4 2023-07-05 00:00:00 5 2023-07-06 00:00:00 6 2023-07-07 00:00:00 7 2023-07-08 00:00:00 8 2023-07-09 00:00:00 9 2023-07-10 00:00:00 10</pre>

Conclusion

In this post, we showed you how to handle the most common date related migration issues when moving from Oracle to Amazon RDS for PostgreSQL or Amazon Aurora PostgreSQL-Compatible Edition. For more information about AWS database migration, refer to [AWS DMS](#) and [AWS SCT](#). You can also refer to [Migration tips for developers for Oracle or Microsoft SQL Server to PostgreSQL](#) and [Migration playbook Oracle to Aurora PostgreSQL](#).

We invite you to leave your feedback in the comments sections.

About the authors

Neha Sharma is working as database specialist with Amazon Web Services. She enables AWS customers to migrate their databases to AWS Cloud. Besides work, she likes to be actively involved in various sports activities and like's to socialize with people.



Sweta Krishna is a Database Migration Specialist with the Professional Services team at Amazon Web Services. She works closely with the customers to help them migrate and modernize their database solutions to AWS.

Comments

2 Comments

Login

G

Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS



Name



Share

Best Newest Oldest

N

NS

5 months ago

Excellent Article. Well done

o o Reply Share

D

Dipendra Singh Chauhan

5 months ago

Amazing blog and explained in very detail with examples. Way to go !!

o o Reply Share

Subscribe

Privacy

Do Not Sell My Data